Python para Negocios 🐍

Programación aplicada a Administración de Empresas

Introducción a Python · Administración de Empresas

¿Qué es Python?

Python es un **lenguaje de programación** que le permite a las computadoras seguir instrucciones que tú escribes, en un lenguaje casi igual al inglés.

Sin Python 😰

- Cálculos manuales y propensos a error
- Hojas de Excel con fórmulas complejas
- Copiar y pegar datos entre archivos
- Reportes que tardan horas en preparar

Con Python ✅

- Cálculos automáticos en segundos
- Análisis de miles de registros
- Reportes generados con un clic
- Decisiones basadas en datos reales

Python es el lenguaje más usado en el mundo para análisis de datos, finanzas, automatización y negocios. Lo usan empresas como Google, Netflix y bancos internacionales.

¿Para qué lo usamos en los negocios?

📊

Análisis de Ventas

Calcular totales, promedios, máximos y mínimos de cualquier período automáticamente.

👥

Gestión de Personal

Nóminas, bonos, clasificación de empleados y cálculo de impuestos de forma automática.

💰

Finanzas

Cálculo de IVA, márgenes, descuentos, presupuestos y proyecciones financieras.

📦

Inventario

Rastrear stock, alertas de reorden y valoración de inventario en tiempo real.

📈

Reportes

Generar reportes de ventas, desempeño y KPIs automáticamente cada semana.

🤖

Automatización

Tareas repetitivas que hoy toman horas, automatizadas en segundos.

Los 6 Conceptos Clave

Variables
Guardar información

Listas
Colecciones de datos

Diccionarios
Registros con nombre

Condicionales
Tomar decisiones

Ciclos
Repetir acciones

Funciones
Código reutilizable

📦 Variables — Guardar Información

Una **variable** es como una caja con etiqueta: le das un nombre y guardas un valor dentro. Python detecta el tipo de dato automáticamente.

```python
# Texto (str) — entre comillas
nombre_empresa = "Tech Store"
gerente        = "Ana García"

# Número entero (int)
empleados      = 42
año_fundacion  = 2018

# Número decimal (float)
ventas_mes     = 85000.50
tasa_iva       = 0.16

# Verdadero / Falso (bool)
activo         = True
en_quiebra     = False

print("Empresa:", nombre_empresa)
print("Empleados:", empleados)
```

Los 4 tipos básicos

Tipo	¿Qué guarda?	Ejemplo
`int`	Números enteros	`42` , `2018`
`float`	Números decimales	`0.16` , `85000.5`
`str`	Texto	`"Ana García"`
`bool`	Verdadero/Falso	`True` , `False`

💡 Los nombres de variables no pueden tener espacios. Usa guión bajo: `ventas_mes`

📦 Variables — Como Calculadora de Negocios

```python
# Datos de un producto
precio_venta   = 1200
costo_unitario = 750
unidades       = 45

# Python hace los cálculos automáticamente
ganancia_unit  = precio_venta - costo_unitario       # 450
ganancia_total = ganancia_unit * unidades             # 20,250
margen         = (ganancia_unit / precio_venta) * 100 # 37.5%

print(f"Ganancia por unidad: ${ganancia_unit}")
print(f"Ganancia total:      ${ganancia_total:,}")
print(f"Margen de utilidad:  {margen:.1f}%")
```

```
Salida:
Ganancia por unidad: $450
Ganancia total: $20,250
Margen de utilidad: 37.5%
```

+ Suma	- Resta	* Multiplicar	/ Dividir

📋 Listas — Colecciones de Datos

Una **lista** guarda varios valores en orden dentro de corchetes `[ ]`. Cada elemento tiene un número de posición (**índice**) que empieza en **0**.

```python
productos = ["Laptop", "Mouse", "Teclado", "Monitor"]
precios   = [15000, 450, 800, 3200]

# Acceder por posición (empieza en 0)
print(productos[0])    # "Laptop"
print(productos[-1])   # "Monitor" (último)

# Operaciones básicas
print(len(productos))  # 4 (cuántos hay)
productos.append("Webcam")  # agregar
print("Mouse" in productos) # True
```

Visualización de una lista

```
productos = ["Laptop", "Mouse", "Teclado", "Monitor"]

┌──────────┬──────────┬──────────┬──────────┐
│ "Laptop" │ "Mouse"  │"Teclado"│"Monitor" │
└──────────┴──────────┴──────────┴──────────┘
    [0]        [1]       [2]       [3]
                                    ↑
                              También [-1]
```

💡 El primer elemento siempre es `[0]`, nunca `[1]`.

🗂️ Diccionarios — Registros con Nombre

Un **diccionario** guarda pares de **clave: valor** entre llaves `{ }`. Perfecto para representar un registro como una ficha de empleado o un producto.

```python
empleado = {
    "nombre":        "Ana García",
    "departamento": "Ventas",
    "salario":       18500,
    "activo":        True
}

# Acceder por nombre de clave
print(empleado["nombre"])      # Ana García
print(empleado["salario"])     # 18500

# Modificar un valor
empleado["salario"] = 20000

# Agregar un campo nuevo
empleado["correo"] = "ana@empresa.com"
```

Visualización de un diccionario

```
empleado = { ... }

┌──────────────────┬──────────────────┐
│ 🔑 Clave         │ 📄 Valor         │
├──────────────────┼──────────────────┤
│ "nombre"         │ "Ana García"     │
│ "departamento"   │ "Ventas"         │
│ "salario"        │ 18500            │
│ "activo"         │ True             │
└──────────────────┴──────────────────┘
```

Lista → posición `[0]` | Diccionario → nombre `["clave"]`

🔀 Condicionales — Tomar Decisiones

Con `if` / `elif` / `else` el programa elige qué hacer según una condición. La **indentación** (4 espacios) es obligatoria — define qué código pertenece a cada rama.

```
ventas_mes = 92000
meta       = 100000
porcentaje = (ventas_mes / meta) * 100

if porcentaje >= 100:
    resultado = "¡Meta superada! 🎉"
    bono      = "10%"
elif porcentaje >= 80:
    resultado = "Cerca de la meta 👍"
    bono      = "5%"
else:
    resultado = "Meta no alcanzada 😔"
    bono      = "0%"

print(f"Logro: {porcentaje:.1f}%  →  {resultado}  →  Bono: {bono}")
```

```
Salida: Logro: 92.0% → Cerca de la meta 👍 → Bono: 5%
```

🔀 Condicionales — Diagrama de Flujo

```
                  ventas >= meta?
                        |
        ┌───────────────┼────────────────┐
       SÍ          NO (pero ≥80%)  NO (< 80%)
        |               |                |
"¡Meta superada!"  "Cerca de        "Meta no
 Bono: 10%          la meta"        alcanzada"
                    Bono: 5%        Bono: 0%
```

✅

if porcentaje >= 100

Primera condición

🤔

elif porcentaje >= 80

Si la primera no se cumplió

❌

else

Si ninguna se cumplió

Operador	Significado	Ejemplo
`==`	Igual a	`estado == "activo"`

🔄 Ciclos (for) — Automatizar Tareas Repetitivas

El ciclo `for` recorre cada elemento de una lista y ejecuta el mismo código para cada uno. En lugar de escribir 100 líneas, escribes 3.

```python
# Sin ciclo — tedioso y propenso a error
print("Laptop:   $15,000")
print("Mouse:    $450")
print("Teclado:  $800")
# ... y así para cada producto
```

❌ Repetitivo, difícil de mantener

```python
# Con ciclo — elegante y escalable
productos = ["Laptop", "Mouse", "Teclado"]
precios   = [15000,    450,     800]

for i in range(len(productos)):
    print(f"{productos[i]}: ${precios[i]:,}")
```

✅ Funciona para 3 o 3,000 productos

```python
# Ciclo con acumulador — calcular total de ventas
ventas = [3200, 4800, 2900, 5100, 4400, 3700, 6200]
total  = 0

for venta in ventas:
    total += venta          # total = total + venta

print(f"Total semanal: ${total:,}")   # Total semanal: $30,300
print(f"Promedio diario: ${total / len(ventas):,.0f}")
```

🔄 Ciclos — Paso a Paso

¿Cómo funciona `for venta in ventas` ?

```
ventas = [3200, 4800, 2900, 5100, 4400]

Iteración 1:  venta = 3200  →  total = 0 + 3200 = 3200
Iteración 2:  venta = 4800  →  total = 3200 + 4800 = 8000
Iteración 3:  venta = 2900  →  total = 8000 + 2900 = 10900
Iteración 4:  venta = 5100  →  total = 10900 + 5100 = 16000
Iteración 5:  venta = 4400  →  total = 16000 + 4400 = 20400
                                                       ↑
                                                Resultado final
```

Forma simple

```python
# Solo necesitas el valor
for producto in productos:
    print("✔", producto)
```

Con índice

```python
# Necesitas posición Y valor
for i in range(len(productos)):
    print(i, productos[i])
```

⚙️ Funciones — Código Reutilizable

Una **función** es un bloque de código con nombre propio que puedes usar muchas veces. Se define con `def`, recibe **parámetros** y devuelve un resultado con `return`.

```
def calcular_precio_final(precio_base, descuento=0):
    precio_desc = precio_base * (1 - descuento)   # aplicar descuento
    iva         = precio_desc * 0.16               # calcular IVA 16%
    total       = precio_desc + iva
    return round(total, 2)

# Llamar la función con distintos productos
laptop  = calcular_precio_final(12000, descuento=0.10)
mouse   = calcular_precio_final(400)
teclado = calcular_precio_final(700, descuento=0.05)

print(f"Laptop  (10% desc. + IVA): ${laptop:,}")    # $12,528.0
print(f"Mouse   (sin desc. + IVA): ${mouse:,}")      # $464.0
print(f"Teclado ( 5% desc. + IVA): ${teclado:,}")   # $773.64
```

💡 **Ventaja clave:** Si el IVA cambia de 16% a 18%, solo cambias una línea en la función — no en cada producto.

⚙️ Funciones — Anatomía

```
#       ① nombre de la función
#               ② parámetros (entrada)
#               ③ valor por defecto
def calcular_precio_final(precio_base, descuento=0):
    #
    # ④ cuerpo (indentado 4 espacios)
    precio_desc = precio_base * (1 - descuento)
    iva         = precio_desc * 0.16
    total       = precio_desc + iva
    #
    return round(total, 2)  # ⑤ resultado (salida)

# ⑥ llamar la función
resultado = calcular_precio_final(12000, 0.10)
```

Las 6 partes

#	Parte	Descripción
①	Nombre	Verbo descriptivo, snake_case
②	Parámetros	Datos que recibe
③	Valor por defecto	Si no se pasa, usa este
④	Cuerpo	El código a ejecutar
⑤	`return`	El resultado que devuelve
⑥	Llamada	Cómo se usa la función

Resumen — Los 6 Conceptos

Concepto	¿Qué es?	Ejemplo de negocio
📦 **Variables**	Caja que guarda un valor	`ventas_mes = 85000`
📋 **Listas**	Varios valores en orden	`["Laptop", "Mouse", "Teclado"]`
🗂️ **Diccionarios**	Registro clave → valor	`{"nombre": "Ana", "salario": 18500}`
🔀 **Condicionales**	Tomar decisiones (if/elif/else)	Clasificar cliente según compras
🔁 **Ciclos**	Repetir para cada elemento	Calcular total de ventas diarias
⚙️ **Funciones**	Código reutilizable con nombre	`calcular_precio_final(precio, desc)`

Variables + Operadores

Listas + Diccionarios

Ciclos + Funciones

Ejercicios Prácticos

Resuelve estos ejercicios en el **Python Trainer**. ¡Usa los ejemplos del menú izquierdo como guía!

Nivel 1 — Variables 📦

1. Crea variables para una empresa: nombre, empleados, ventas del mes
2. Calcula la ganancia: `precio_venta = 800`, `costo = 500`, `unidades = 30`
3. Calcula el precio con IVA 16% de un producto de $1,200

Nivel 2 — Listas y Diccionarios 📋🗂️

4. Crea una lista con 5 productos de tu empresa. Imprime el primero y el último.
5. Crea un diccionario con tu información: nombre, carrera, semestre, ciudad

Nivel 3 — Condicionales 🔀

7. Dado `descuento_anual = 65000`, clasifica al cliente:
   - ≥ 100,000 → VIP (15%)
   - ≥ 50,000 → Premium (10%)
   - ≥ 20,000 → Regular (5%)
   - Menos → Nuevo (0%)

Nivel 4 — Ciclos y Funciones 🔄⚙️

8. Recorre esta lista e imprime cada producto con su precio: `precios = [3200, 4800, 2900, 5100, 4400]`
9. Crea una función `calcular_iva(precio)` que devuelva el precio con 16% de IVA

Cómo Usar el Python Trainer

📋

1. Panel izquierdo

Selecciona un **tema** y carga ejemplos predefinidos con un clic.

⌨️

2. Editor central

Escribe o modifica el código. Ejecuta con `Ctrl` + `Enter` o el botón ▶ **Ejecutar**.

💡

3. Tabs de resultado

Resultado — ve el output de tu código.
Concepto Visual — diagrama del tema.

Consejos:

- • La tecla `Tab` inserta 4 espacios (indentación)
- • Si hay un error, lee el mensaje — te dice en qué línea está
- • Modifica los ejemplos y experimenta — no puedes romper nada

¡A practicar! 🐍

Abre el **Python Trainer** y empieza con los ejercicios del Nivel 1.

Recuerda: la única manera de aprender a programar es **escribiendo código**. Los errores son parte del proceso — cada error es una lección.